

Análisis y Diseño con Patrones

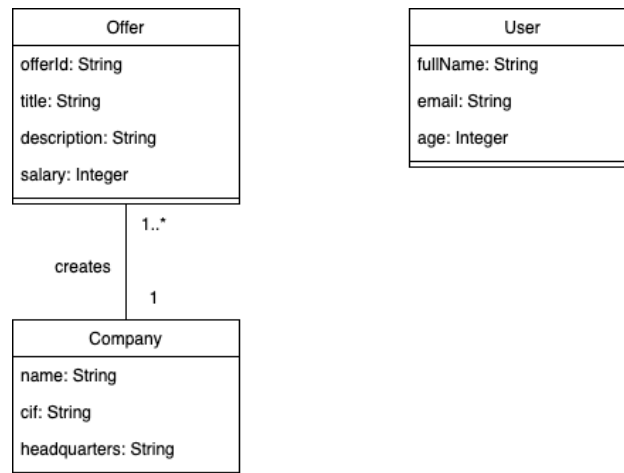
PEC 2: Patrones de Diseño y de Asignación de Responsabilidades (evaluada sobre 50 puntos)

Pregunta 1 (12,5 puntos)

Enunciado

Nos encontramos a finales de 2002, justo al acabar el *dot-com bubble*. Unos inversores se han reunido con nosotros para que les ayudemos a desarrollar la idea que tienen en mente. En concreto, se han dado cuenta de que la gente empieza a consultar internet para buscar ofertas de trabajo. Por ello, deciden crear un portal de empleo online en el que los usuarios puedan consultar las distintas ofertas de trabajo existentes. Inicialmente, el sistema solo tiene usuarios, compañías y ofertas. Sin embargo, estas tres entidades se relacionan de un modo sencillo (estamos en los primeros días de la idea).

En concreto, tenemos una clase `User`, la cual se utiliza para representar a los usuarios que consultan la web. Deben estar registrados, y para ello se pide un nombre completo (*fullName*), email (*email*) y una edad (*age*). Las compañías (`Company`) contienen un nombre (*name*), un CIF (*CIF*) y la ciudad en la que tienen su sede central (*headquarters*). Las ofertas (`Offer`) son creadas por las empresas y tienen un id para identificarlas (*offerId*) un título (*title*), una descripción (*description*) y un salario fijo (*salary*). El diagrama que ilustra este sencillo modelo es el siguiente:



Restricciones de integridad

- User se identifica por medio de *email*.
- Offer se identifica por medio de *offerId*.
- Company se identifica por medio de *cif*.

Vamos a añadir una serie de funcionalidades. En concreto, queremos que los usuarios puedan inscribirse en esas ofertas de empleo. Para ello, se deben gestionar las distintas candidaturas. Las candidaturas se identifican tanto por el usuario que postula (User) como por la oferta a la que se presenta (Offer). Además, las candidaturas tienen una fecha de creación (el momento en el que el usuario se presenta a la oferta). Con ello, se debe ofrecer la posibilidad de añadir la candidatura de un usuario a una oferta. Igualmente, dada una oferta, el sistema debe permitir tanto recuperar como borrar la candidatura de un usuario a dicha oferta a partir del identificador de un usuario. De igual forma, dado un usuario, se debe poder borrar y recuperar la candidatura de dicho usuario a una determinada oferta por su identificador de oferta.

Se pide

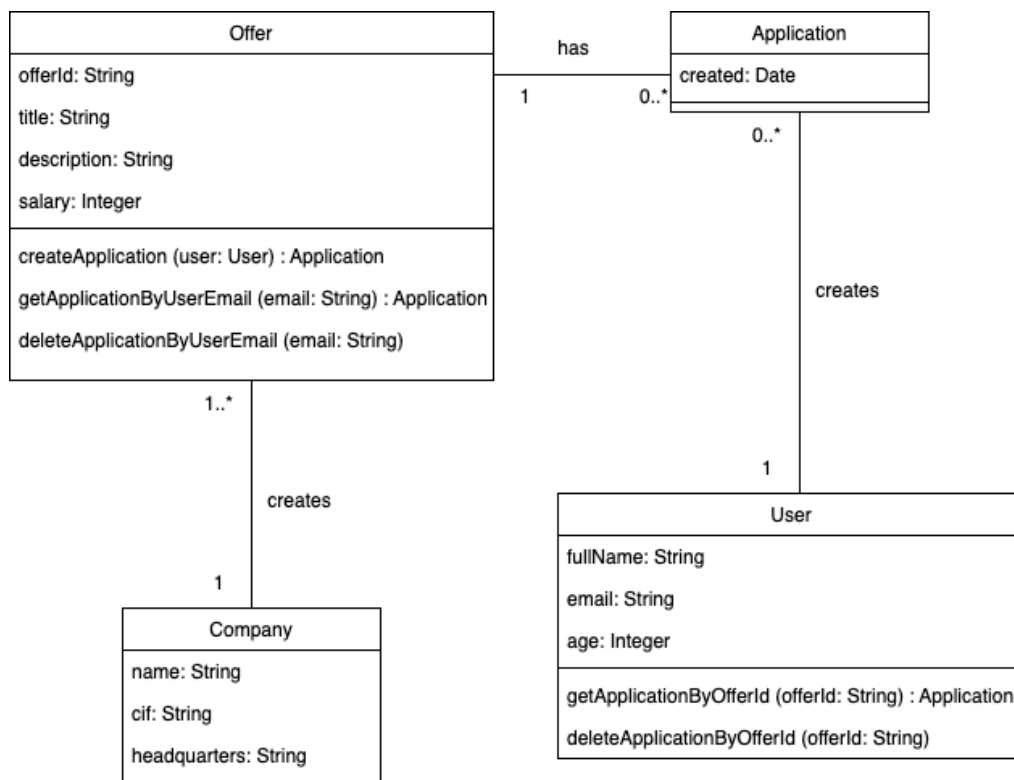
- a) (5 puntos) Justificar qué patrones de responsabilidad debemos tener en cuenta para añadir estas funcionalidades.
- b) (7,5 puntos) Añade el diagrama de clases incluyendo la solución al soporte para candidaturas, incluyendo las clases y operaciones para las funcionalidades descritas.

Nota: No se pide ir mucho más allá en cuanto a funcionalidades. Es decir, durante toda la PEC usaremos el mismo caso (portal de empleo), por lo que no es necesario implementar funcionalidades que no se piden en el enunciado.

Solución

a) Los patrones más adecuados para este caso son dos. En primer lugar, el patrón **Creador**. Este patrón se puede implementar de dos maneras en este ejercicio, en Offer y en User. Tanto Offer::createApplication(user : User) como User::createApplication(application : Application) serían opciones válidas, dado que realmente no existe composición entre Offer y Application, ni entre User y Application, por tanto la relación de Application tanto con User como Offer es "simétrica". Junto a ello, es necesario tener en cuenta el patrón **Experto**. En nuestro caso, Offer actuará como experto cuando sea el identificador de usuario (email) el parámetro que utilizamos para la búsqueda/borrado de Application. Por otro lado, User actuará como Experto cuando el parámetro de búsqueda/borrado de Application sea el offerId.

b) El diagrama estático de clases es el siguiente (considerando el creador en la Offer):



Restricciones de integridad

- *Application* se identifica por medio de *email* y *offerId*.

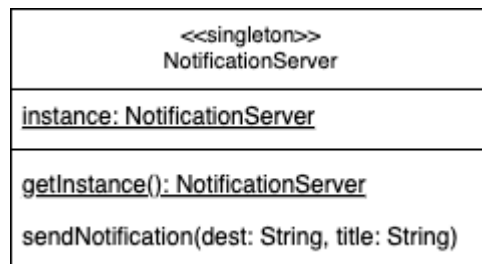
Es necesario destacar que *createApplication* sí necesita el e-mail, ya que necesitamos saber cuál es el usuario que desea postular para crear la relación. Además, en la operación *createApplication*, la fecha de creación de una *Application* es autogenerada, es decir, se crea en el momento de ejecución de dicho método.

Pregunta 2 (15 puntos)

Enunciado

Saltamos al año 2008, cuando una importante compañía tecnológica informa de que ha desarrollado un sistema de notificaciones para sus dispositivos móviles. Los inversores quieren que aprovechemos esta novedad en nuestro sistema de búsqueda de empleo online y por ello quieren que añadamos a nuestros usuarios un servicio de notificaciones. De este modo, los usuarios podrán escoger sus compañías favoritas. Cuando alguna de ellas crea alguna oferta, se les enviará una notificación push en su dispositivo móvil con el título de la oferta si tienen nuestra aplicación instalada, en base al email del usuario.

Disponemos de un servicio externo en un servidor llamado *NotificationServer*. No es necesario conocer todos los detalles de su configuración ni de los métodos que contiene. Estos detalles quedan encapsulados en esta clase, a cuyos servicios se accede a través de la aplicación del patrón Singleton:



Se pide:

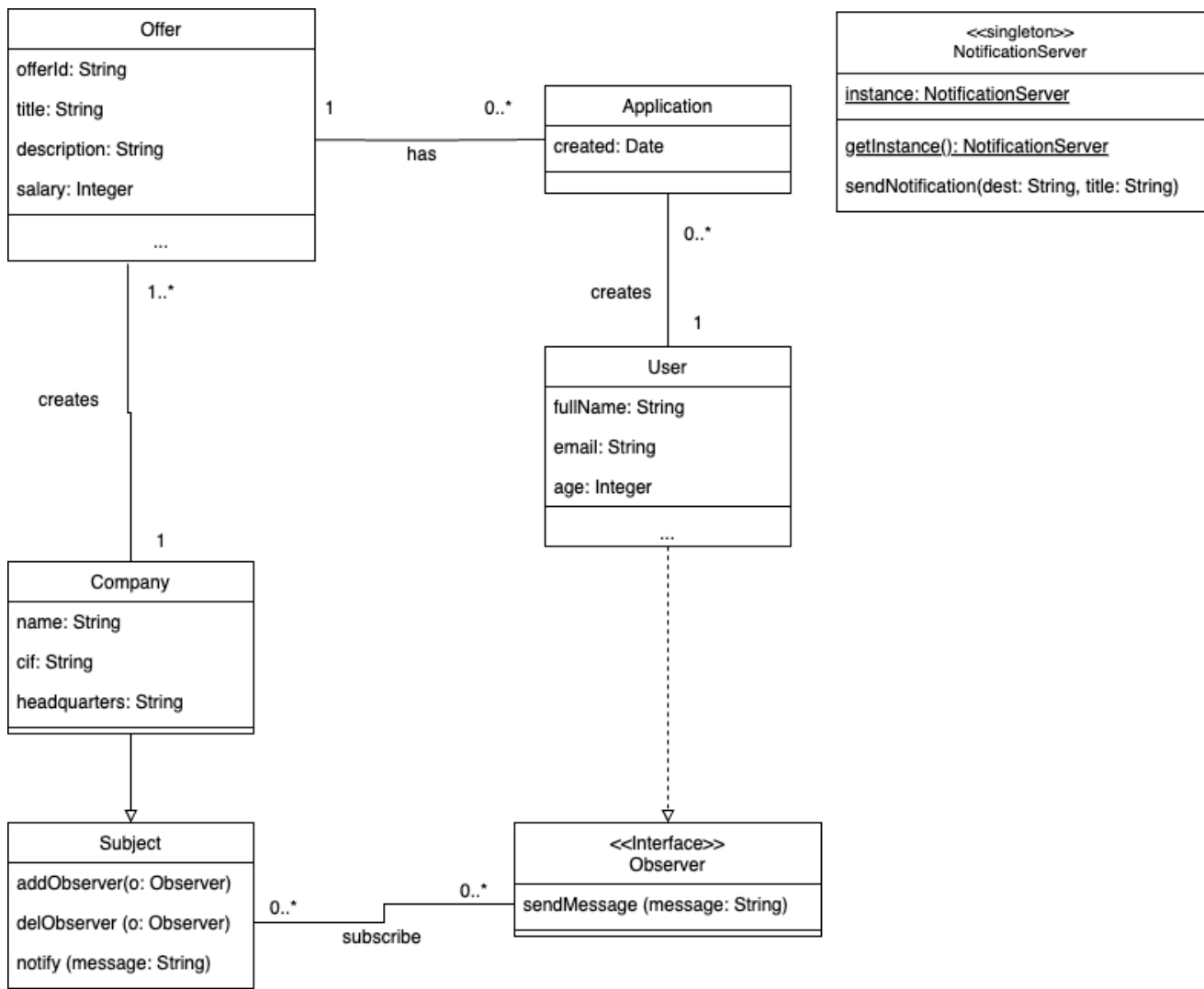
- (5 puntos) Justificar qué patrones de diseño podemos aplicar para gestionar las notificaciones.
- (5 puntos) Modificar y presentar el diagrama estático de clases
- (5 puntos) Mostrar el pseudocódigo de las clases y operaciones que intervienen en el servicio de notificación a los usuarios.

Nota: No hay que tener en cuenta si el usuario tiene o no la aplicación instalada en un dispositivo. Este ejercicio únicamente busca la solución para el caso concreto del problema que planteamos.

Solución

a) El patrón que mejor se ajusta a este caso es el patrón **Observer**. Aunque el contexto teórico dice que se aplica cuando "un cambio en un objeto implica cambios en otros objetos", en los Módulos se ha mostrado que se puede aplicar a la gestión de estas notificaciones a las suscripciones. En este caso específico utilizando la primera de sus variantes, donde el objeto observado (Company) no se pasa a sí mismo como parámetro al servicio que realiza la notificación. Company notifica a su lista de suscriptores el título de cada nueva oferta que crea.

b) El diagrama de clases estático que se propone como solución es el siguiente:



- c) El pseudocódigo de las clases que intervienen en el servicio de notificaciones a los usuarios es el siguiente:

```
public class Subject {
    protected List<Observer> observers;

    public void addObserver (Observer o) {
        observers.add(o);
    }

    public void delObserver (Observer o) {
        observers.remove(o);
    }

    public void notify (String message) {
        foreach(observer in observers) {
            observer.sendMessage(message);
        }
    }
}

public class Company extends Subject {
    private List<Offer> offers;
    public void addOffer (Offer offer) {
        offers.add(offer);
        String title = offer.getTitle();
        notify(title);
    }
}

public class User implements Observer {
    public void sendMessage (String message) {
        NotificationServer server = NotificationServer.getInstance();
        server.sendNotification(this.getEmail(),"New offer from your
        favorite Company ");
    }
}
```



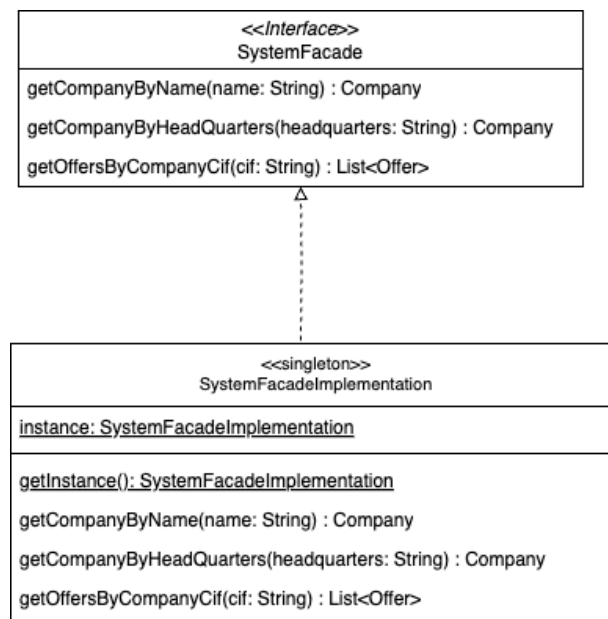

Pregunta 3 (12,5 puntos)

Enunciado

En esta ocasión, pasamos al año 2014. Nos encontramos en pleno boom de los datos y por ello, nos han pedido que seamos capaces de sacar el dato de candidaturas medias por oferta para una compañía determinada. Es decir, buscamos hallar el número medio de personas que se han presentado a las ofertas de una empresa. Para entenderlo mejor, supongamos que una compañía A tiene 2 ofertas. Una de ellas tiene 10 candidaturas y otra tiene 20 candidaturas. En este caso, el valor medio sería 15.

Esta media se calcula de manera diaria a las 00:00, de tal modo que cada día podamos tener el dato actualizado. De este modo, en la interfaz de usuario, priorizaremos las ofertas que pertenecen a compañías con más candidaturas medias por oferta. Tenemos que desarrollar un servicio de la capa de dominio que permita al sistema calcular dicho valor.

En esta ocasión, es importante tener en cuenta que *Offer* tiene una asociación que es un objeto de tipo *Company*, que refleja la compañía que la ha creado. Además, es importante recordar que *Offer* tiene una lista de *Application*. Destacar además, que para la resolución de esta tarea no se pueden añadir nuevos métodos a las clases existentes. Sin embargo, se puede utilizar la siguiente clase *Facade* del dominio para acceder a las entidades del sistema:



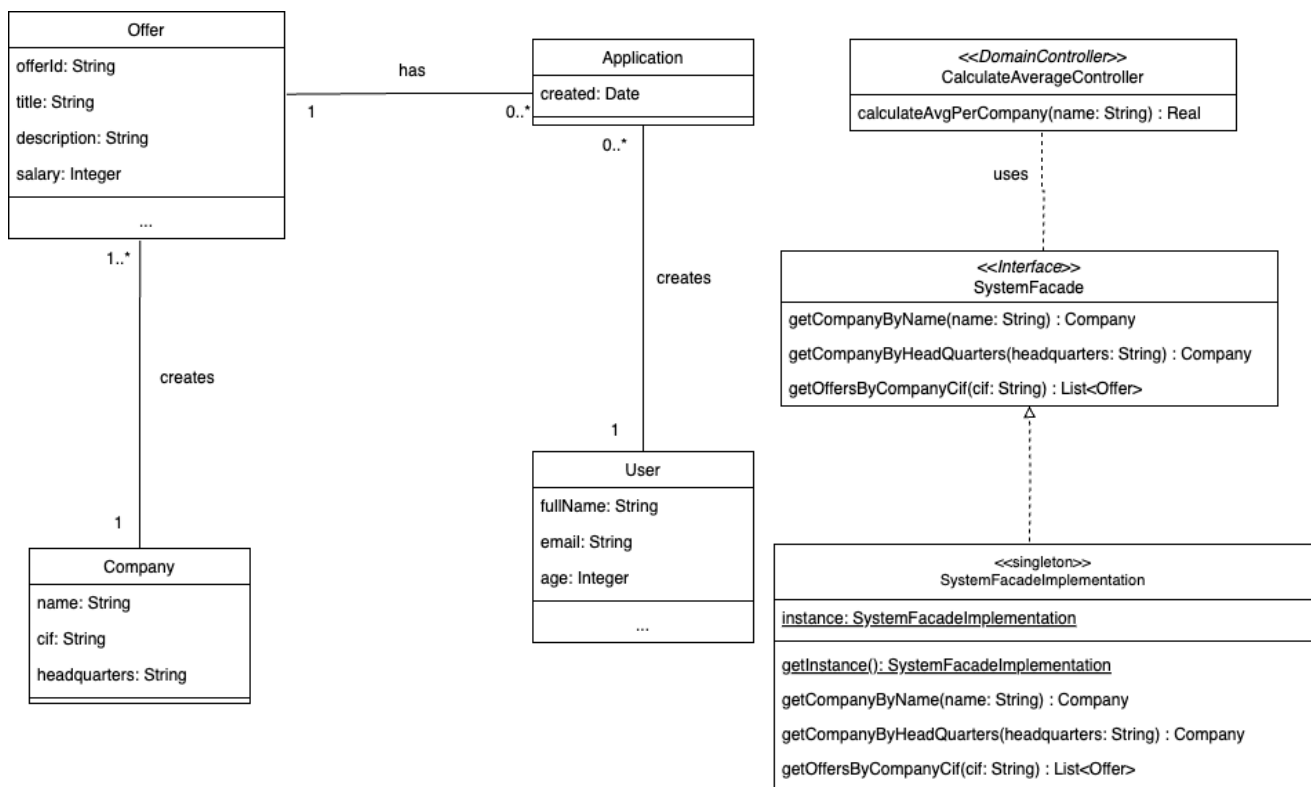
Se pide:

- a) (2.5 puntos) Justifica qué patrones de diseño se tienen que aplicar para incorporar esta nueva funcionalidad.
- b) (5 puntos) Muestra el diagrama de clases con esta funcionalidad nueva.
- c) (5 puntos) Muestra el diagrama de secuencia para esta funcionalidad.

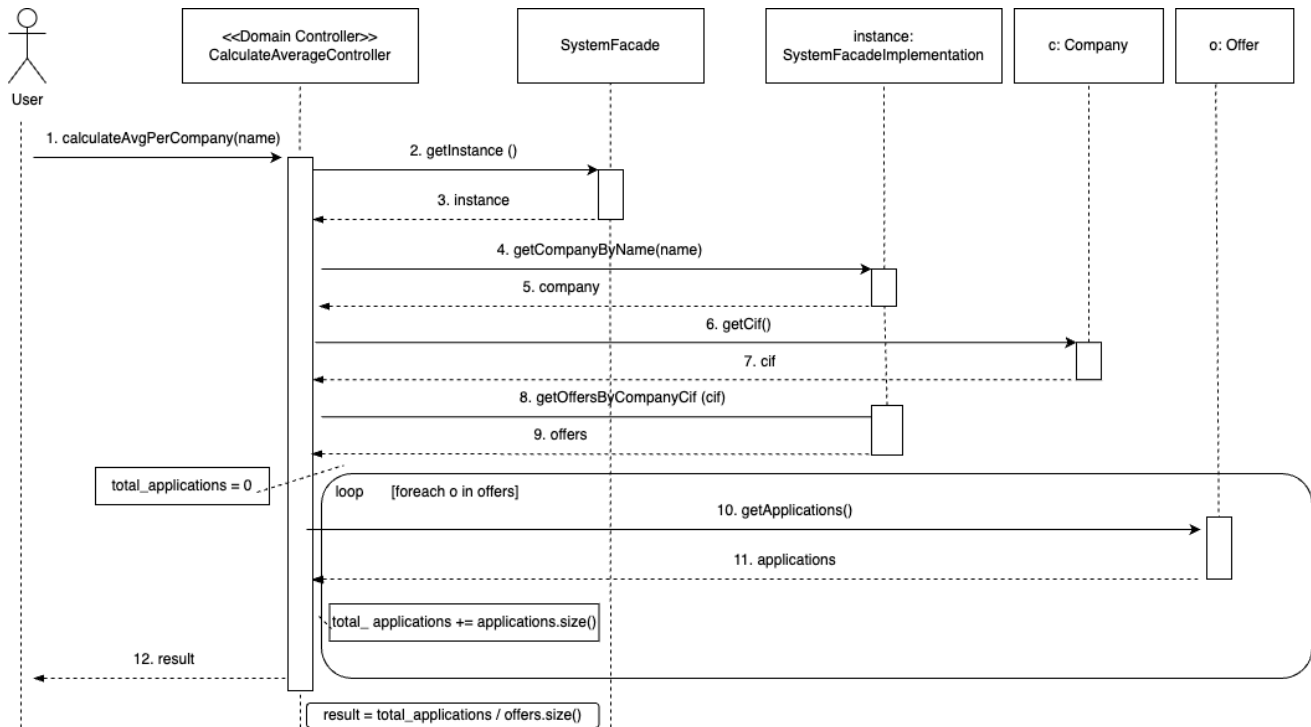
Solución

- a) Para este problema en concreto podemos optar por utilizar el patrón **Controlador**. En este punto necesitamos una clase que asuma el control de gestionar la capa de dominio y hacer las comprobaciones necesarias para calcular la media. Este controlador recibirá el título de la empresa y a partir de ahí, usando las operaciones de *SystemFacade* será capaz de tomar primero la empresa, para después poder obtener la lista de ofertas que han sido creadas por esa empresa. Tras ello, podrá obtener la lista de candidaturas y así calcular la media de candidaturas.

- b) El diagrama estático de clases que se propone como solución es el siguiente:



c) El diagrama de secuencia para esta funcionalidad es el siguiente:



Primeramente, se realiza una llamada al método controlador `CalculateAverageController::CalculateAvgPerCompany (name)`, donde `name` es el nombre de la compañía a la que le vamos a calcular la media de todas las ofertas. Tras ello, obtenemos una instancia del singleton `SystemFacadeImplementation`. Tras ello obtenemos una compañía a través de su nombre por medio de `SystemFacadeImplementation::getCompanyByName` y una vez tenemos la compañía, podemos llamar a `SystemFacadeImplementation::getOffersByCompanyCif` con el cif de la compañía que hemos obtenido previamente para obtener todas las ofertas de la compañía. Tras tener todas las ofertas, para cada una de ellas calculamos el tamaño de la lista de `Applications` que tiene. Sumamos todos esos valores y los dividimos por el número de ofertas que hay, de tal modo que tenemos el número medio de candidaturas por oferta para una compañía.

Pregunta 4 (10 puntos)

Enunciado

Llegamos a la actualidad, año 2024. Hasta el día de hoy las empresas nunca cerraban sus ofertas y los usuarios seguían aplicando a dichas ofertas aunque estuviesen cerradas, porque el sistema no hacía ningún tipo de actualización. Para solventar este problema, se requiere que diseñemos una solución que permita modelar cuando una oferta está abierta y cuando está cerrada. Es decir, debemos permitir que el sistema pueda abrir y cerrar ofertas de un modo sencillo.

Además:

- Se lanza la excepción *OfferAlreadyClosed* si se intenta cerrar una oferta que ya está cerrada.
- Se lanza la excepción *OfferAlreadyOpen* si se intenta abrir una oferta que ya está abierta.
- Una oferta cerrada puede volver a ser abierta. Esto puede ocurrir cuando un trabajador no supera el periodo de prueba y la empresa decide reabrir la posición. No hay que modelar este último concepto de periodo de prueba, simplemente es para ilustrar que una oferta cerrada se puede volver a abrir.
- Cuando se cierra una oferta es porque una determinada candidatura ha sido elegida. Es decir, no se van a cerrar ofertas que hayan quedado desiertas.
- Buscamos guardar la fecha de apertura y cierre de las ofertas, así como la candidatura seleccionada.

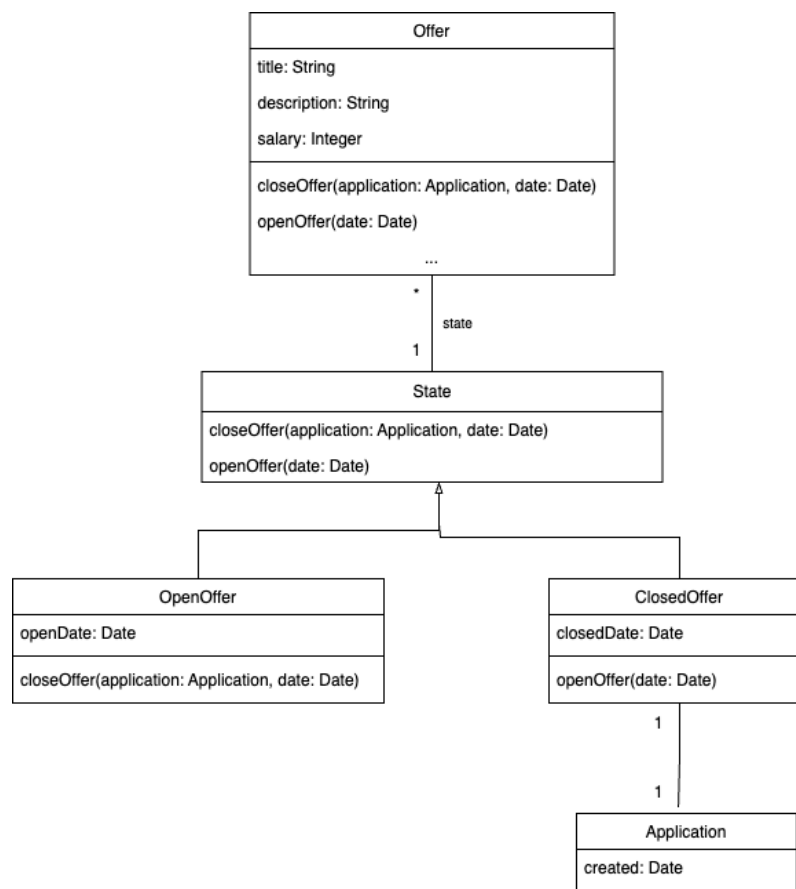
Nota: No es necesario incluir todo el diagrama estático de los ejercicios previos, simplemente la parte que consideres que aporta a la resolución del problema.

Se pide:

- a) (3 puntos) Justifica qué patrón de diseño aplicarías.
- b) (7 puntos) Proporciona el diagrama de clases con la aplicación del patrón.

Solución

- a) El patrón que mejor funciona aquí es el patrón **State**, ya que con él vamos a poder modelar de un modo sencillo los dos posibles estados de la oferta y vamos a ser capaces de definir los distintos métodos que permiten transicionar de uno a otro.
- b) El diagrama estático de la aplicación del patrón estado es el siguiente:



Ahora, gracias al patrón State, podemos transicionar entre abierto y cerrado. Si intentamos abrir una oferta cerrada, como será **ClosedOffer**, saltará una excepción diciendo que ya está cerrada. Por otro lado, si intentamos abrir una ya abierta, saltará otra excepción diciendo que ya está abierta. En caso de que no salten las excepciones, será porque la transición entre estados sí que estará permitida y así tendremos el sistema actualizado correctamente.

Restricciones de Integridad:

- La fecha de cierre de una oferta siempre será igual o mayor a la fecha de creación de una oferta.